

DocTurn

Hospital Patient Assignment & Secure Messaging Platform

Engineering Specification

Document type	Deep technical specification (engineering audience)
Version	1.0
Date	June 05, 2026
Status	Confidential — internal engineering reference
System	Multi-tenant web application + Expo React Native mobile app

This document is a complete engineering reference for the DocTurn platform. It describes the system architecture, technology stack, full data model, REST and WebSocket API surface, authentication and authorization model, multi-tenancy strategy, real-time messaging, the patient-assignment workflow, external service integrations, the web and mobile frontends, the HIPAA security posture, and the deployment topology.

Contents

Contents	2
1. System Overview	5
1.1 Primary user roles	5
1.2 Core capabilities	5
1.3 Document scope	5
2. Architecture	7
2.1 High-level component diagram	7
2.2 Request lifecycle	7
2.3 Design principles	8
3. Technology Stack	9
3.1 Backend	9
3.2 Frontend (web)	9
3.3 Mobile	9
3.4 Data & infrastructure	10
4. Data Model	11
4.1 Modeling conventions	11
4.2 Core clinical tables	11
4.3 Scheduling & resource tables	11
4.4 Messaging & directory tables	12
4.5 Security, compliance & session tables	12
4.6 Integration & CMS tables	13
4.7 Assignment status state machine	13
5. API Surface	14
5.1 Authentication & users	14
5.2 Providers, directory & rotation	14
5.3 Patients & assignments	14

5.4 Messaging	15
5.5 Organizations, developer, CMS & SMS	15
5.6 Health & mobile	16
5.7 WebSocket server	16
6. Authentication & Authorization	17
6.1 Authentication	17
6.2 Multi-factor authentication	17
6.3 Role-based access control	17
6.4 Replit OIDC note	17
7. Multi-Tenancy	19
8. Patient Assignment Workflow	20
8.1 End-to-end flow	20
8.2 Round-robin rotation	20
8.3 Expiration & reassignment	20
9. External Integrations	21
9.1 Twilio (SMS & MFA)	21
9.2 OpenAI (AI assistance)	21
9.3 Firebase (push notifications)	21
9.4 Amion (scheduling)	21
10. Frontend Architecture	22
10.1 Web SPA	22
10.2 Role dashboards	22
10.3 Mobile app	22
11. Security & HIPAA Compliance	23
11.1 Transport & HTTP hardening	23
11.2 Audit & PHI logging	23
11.3 Session & account controls	23
11.4 Threat-model focus areas	23

12. Deployment & Operations	24
12.1 Environments	24
12.2 Configuration & secrets	24
12.3 Schema changes	24
12.4 Observability	24
13. Appendix	25
13.1 Repository layout	25
13.2 Glossary	25

1. System Overview

DocTurn is a multi-tenant, HIPAA-oriented SaaS platform that coordinates patient assignment and secure clinical communication inside hospitals. Emergency-room (ER) physicians admit patients and hand them off to hospitalists; the platform automates who receives each patient through a configurable round-robin rotation, notifies the selected provider in real time across multiple channels, tracks acknowledgement, and provides a HIPAA-compliant messaging layer for all providers.

The product serves several distinct roles across many independent hospital organizations (tenants). Each tenant's data — users, patients, assignments, messages, audit records, and configuration — is isolated by an `organizationId` that is enforced at the data-access layer.

1.1 Primary user roles

Role	Responsibilities
director	Hospital administrator. Manages providers, shift definitions, round-robin configuration, reassignment, and organization settings.
er_director	ER administrator. Manages ER physicians and oversees ER-side assignment flow.
er_doctor / er_physician	Admits patients, creates assignment requests, and uses AI-assisted patient intake.
hospitalist	Receives patient assignments, accepts/declines, manages census and patient cap, and communicates with peers.
developer	Platform operator. Cross-tenant administration: organizations, users, CMS, integrations, diagnostics, impersonation.

1.2 Core capabilities

- **Patient assignment workflow** — ER physicians create assignments that are routed to hospitalists via round-robin or manual selection, with configurable expiration.
- **Real-time notifications** — WebSocket push, Firebase Cloud Messaging push, and Twilio SMS fallback.
- **Secure messaging** — direct, group, and emergency-broadcast conversations with delivery/read receipts and audit logging.
- **AI assistance** — OpenAI-powered extraction of structured clinical data from free text and natural-language command processing.
- **Multi-tenancy** — organization-scoped data isolation with per-tenant branding, feature flags, and integration credentials.
- **HIPAA controls** — comprehensive audit trail, PHI access logging, short session timeouts, MFA, and encryption in transit.
- **Cross-platform clients** — a React/Vite web SPA and an Expo React Native mobile app sharing one backend, session model, and database.

1.3 Document scope

This specification documents the production application: the Express/TypeScript backend, the PostgreSQL data model, the React web client, and the Expo mobile client. Experimental pages, standalone HTML test harnesses, and developer-only self-test endpoints are noted where relevant but are out of scope for production behavior.

2. Architecture

DocTurn is a TypeScript monolith. A single Express server hosts the REST API and a WebSocket server, serves the compiled React SPA, and talks to PostgreSQL through Drizzle ORM. The Expo mobile app is a separate client that consumes the same API and session model. External services (Twilio, OpenAI, Firebase, Amion) are reached server-side so that secrets never touch the client.

2.1 High-level component diagram

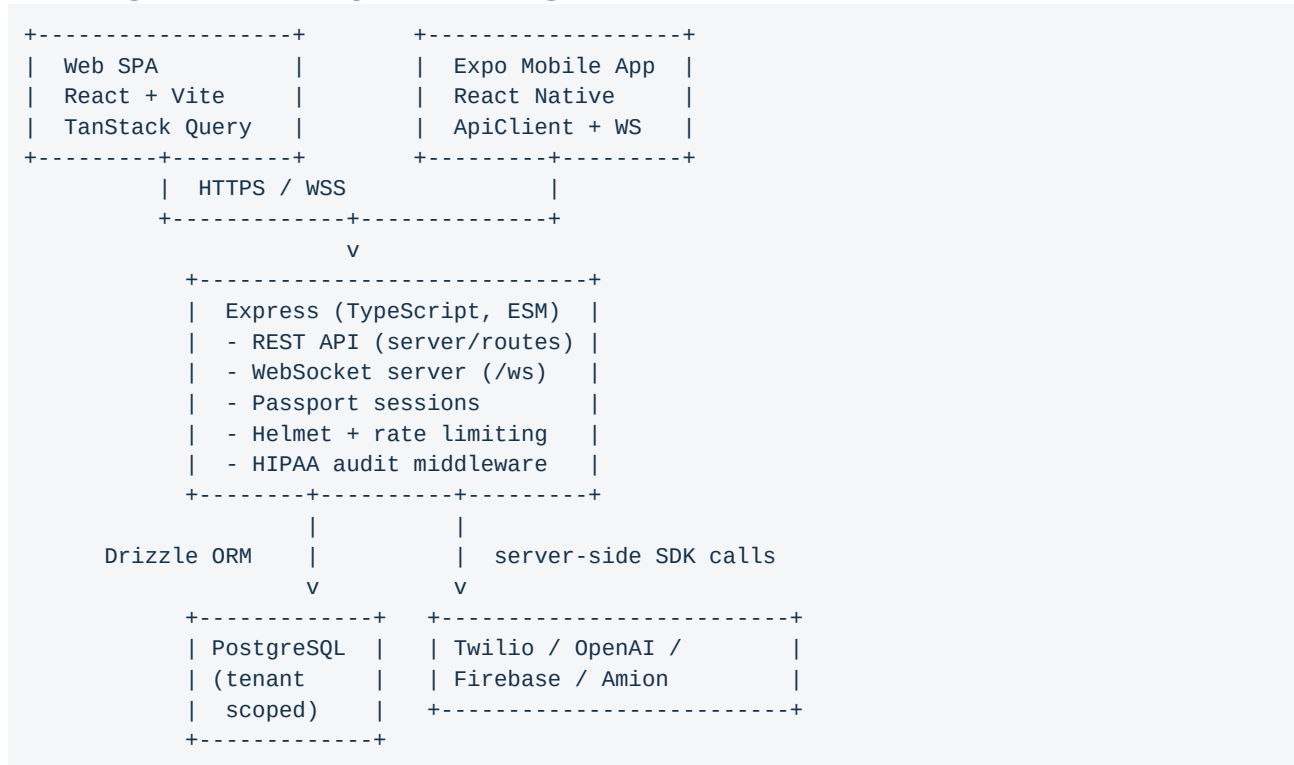


Figure 2.1 — Logical component topology. One backend serves two clients and brokers all third-party calls.

2.2 Request lifecycle

- **Ingress** — TLS terminates at the platform edge; traffic reaches Express on a single port that serves both API and static assets.
- **Security middleware** — Helmet sets HTTP security headers; tiered express-rate-limit guards auth and general API routes.
- **Session resolution** — Passport deserializes the session from the PostgreSQL-backed store using the `docturn.sid` cookie.
- **Authorization** — route handlers assert the caller's role and that the target record matches the caller's `organizationId`.
- **Data access** — handlers call the storage layer (`IStorage`), which executes Drizzle queries scoped by tenant.
- **Audit** — HIPAA middleware and explicit audit inserts record security-relevant events and PHI access.

- **Real-time fan-out** — state changes broadcast over WebSocket to the relevant tenant/users; push and SMS provide out-of-app delivery.

2.3 Design principles

- **Tenant isolation first** — every read and write is scoped by `organizationId`; cross-tenant access is a primary threat.
- **Thin routes, central storage** — business data access lives behind the `IStorage` interface to keep route handlers focused on validation and authorization.
- **Server-authoritative state** — security-sensitive values (assignment routing, expiry, roles) are computed on the server, never trusted from the client.
- **Secrets stay server-side** — third-party SDK calls are made by the backend; clients never receive integration credentials.
- **Graceful degradation** — notifications cascade (WebSocket → push → SMS) and integrations fall back to local behavior when unconfigured.

3. Technology Stack

3.1 Backend

Concern	Technology	Version
Runtime	Node.js (ESM) + tsx (dev)	Node 20
Web framework	Express	^4.21.2
Language	TypeScript	5.x
ORM	Drizzle ORM	^0.45.2
Validation	Zod + drizzle-zod	^3.24 / ^0.7
Auth	Passport (local strategy)	^0.7.0
Real-time	ws (WebSocket)	^8.18.0
AI	openai	^5.23.2
SMS / MFA	twilio	^5.7.0

3.2 Frontend (web)

Concern	Technology	Version
UI library	React	^18.3.1
Build tool	Vite	5.x
Routing	wouter	^3.3.5
Server state	TanStack Query	^5.60.5
Styling	Tailwind CSS + shadcn/ui	3.x
Drag & drop	@dnd-kit	—
Validation	Zod (shared schemas)	^3.24.2

3.3 Mobile

Concern	Technology	Version
Framework	Expo SDK	~52
Runtime	React Native	0.76.9
React	react / react-dom	18.3.1
Navigation	@react-navigation (bottom-tabs)	—
Storage	@react-native-async-storage	1.23.1
Push	expo-notifications	~0.29

Concern	Technology	Version
Web target	@expo/metro-runtime	~4.0.1

3.4 Data & infrastructure

- **Database** — PostgreSQL, accessed via the DATABASE_URL connection string.
- **Schema management** — Drizzle Kit with `npm run db:push` (no hand-written SQL migrations).
- **Sessions** — persisted in PostgreSQL via a connect-pg session store.
- **Dev orchestration** — Replit workflows run the web app (port 5000) and the Expo web build (port 8080) side by side.
- **Production** — multi-stage Docker build, Docker Compose, Nginx reverse proxy, and auto-scaling.

4. Data Model

The schema is defined in `shared/schema.ts` using Drizzle ORM and is shared between the server and (via the Zod insert schemas) the clients. It comprises 37 domain tables plus the session table. The organizations table is the tenancy root; nearly every other table carries an `organizationId` foreign key.

4.1 Modeling conventions

- Each table has a serial `id` primary key, except conversations and messages, which use UUIDs.
- Enumerations are expressed as `text({ enum: [...] })` columns (e.g. assignment status, shift type, risk level).
- Array columns use the column-method form, e.g. `text().array()`.
- For every insertable table, a `createInsertSchema(...).omit({ id, createdAt, updatedAt })` Zod schema and inferred insert/select types are exported.
- JSON-heavy CMS and configuration tables use `jsonb` columns (shift-type definitions, landing/contact page content, portal feature flags).

4.2 Core clinical tables

Table	Key columns	Purpose
organizations	id, name, code, amionApiKey, roundRobinShiftTypes, shiftTypeDefinitions (jsonb), timezone	Tenant root: hospital metadata, rotation config, integration keys.
users	id, organizationId, username, password, role, twoFactorEnabled, replitUserId	All accounts across every role; scrypt-hashed credentials.
hospitalists	id, organizationId, userId, specialty, currentPatientCount, patientCap, rotationOrder, shiftType, shift times	Provider rotation profile for hospitalists and ER physicians.
patients	id, organizationId, initials, roomNumber, issueSummary, assignedHospitalistId, erDoctorId, status	PHI-bearing patient record (initials only, not full names).
assignments	id, patientId, hospitalistId, erDoctorId, status, expiresAt, smsSid	Assignment request lifecycle: pending → accepted/rejected/expired/cancelled.
assignment_messages	id, assignmentId, senderId, content	Lightweight messages attached directly to an assignment.

4.3 Scheduling & resource tables

Table	Purpose
daily_schedules	Per-hospitalist daily shift records (date + shift type).
staff_schedule	Department-level staff shift windows (start/end timestamps).

Table	Purpose
departments	Hospital departments with capacity and alert thresholds.
beds	Bed inventory by department, status, and occupying patient.
equipment	Equipment inventory by department, type, and status.
resource_alerts	Threshold-breach alerts for capacity/resources.
resource_metrics	Time-series resource utilization metrics.
system_settings	Per-organization key/value settings, including round-robin rotation index.

4.4 Messaging & directory tables

Table	Purpose
conversations	UUID-keyed conversation (direct / group / emergency) with a participants array.
messages	UUID-keyed message with content, encryption flag, and conversation FK.
message_delivery_status	Per-recipient delivered/read tracking for each message.
message_threads	Threading metadata for grouped messages.
message_audit_log	Audit trail of messaging actions for HIPAA compliance.
provider_directory	On-call / availability directory entries per provider.
emergency_broadcasts	Broadcast messages with a target-audience array.
broadcast_acknowledgments	Per-user acknowledgement of emergency broadcasts.

4.5 Security, compliance & session tables

Table	Purpose
sessions	PostgreSQL session store backing Passport sessions.
user_sessions	Application-level session/presence records (online state, token).
audit_logs	Primary HIPAA audit trail: action, resourceType, resourceId, details, riskLevel, IP, user agent.
phi_access_logs	Dedicated PHI access trail: patientId, accessType, dataFields accessed.
security_incidents	Recorded security incidents with severity and status.
sms_verification_codes	Short-lived SMS one-time codes for MFA / verification.
pending_registrations	Self-registration requests awaiting director approval.
provider_registrations	Provider onboarding records.
role_types	Configurable role definitions (permissions, portal access as JSON).

4.6 Integration & CMS tables

Table	Purpose
firebase_configurations	Per-organization Firebase project id and service-account credentials.
device_tokens	Registered push device tokens per user/device.
push_notifications	Outbound push notification records and status.
integration_settings	Generic per-organization integration configuration.
portal_configurations	Per-organization/portal feature toggles (AI chatbot, SMS, etc.).
landing_page_settings	CMS content for the marketing landing page (hero, pricing, testimonials, SEO).
contact_page_settings	CMS content for the contact page.

4.7 Assignment status state machine

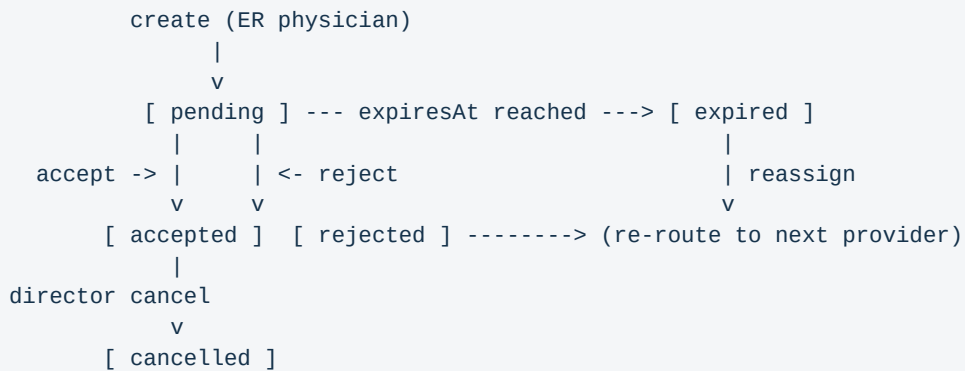


Figure 4.1 — Assignment status transitions. Expiry and reassignment are server-driven; cancellation is a director action.

5. API Surface

The REST API lives primarily in `server/routes.ts` with authentication routes wired in via `setupAuth(app)` from `server/auth.ts`. All non-public routes require an authenticated session; privileged routes additionally assert role and tenant. The tables below group representative endpoints by domain (the full surface is large; duplicates are resolved to the first matching handler by Express).

5.1 Authentication & users

Method & path	Auth	Purpose
POST <code>/api/register</code>	Public	Self-register; creates a pending registration awaiting director approval.
POST <code>/api/login</code>	Public	Username/password login; returns 202 if MFA is required.
POST <code>/api/2fa/complete-login</code>	Public (pending)	Completes the MFA challenge for a pending login.
POST <code>/api/logout</code>	Auth	Destroys the session.
GET <code>/api/user</code>	Auth	Returns the sanitized current user.
GET <code>/api/users</code>	Director	Lists users in the caller's organization.
POST <code>/api/dev/users</code>	Developer	Cross-tenant user creation.

5.2 Providers, directory & rotation

Method & path	Auth	Purpose
GET <code>/api/hospitalists</code>	Auth	List hospitalists in the organization.
GET <code>/api/hospitalists/working</code>	Auth	List providers currently on shift.
POST <code>/api/director/hospitalists</code>	Director / ER director	Create a hospitalist or ER physician account + rotation profile.
DELETE <code>/api/physicians/:id</code>	Director	Delete a provider (blocked if pending assignments exist).
PATCH <code>/api/hospitalists/:id/working-status</code>	Auth	Toggle a provider's on-shift status.
PATCH <code>/api/physicians/:id/capacity</code>	Auth	Update patient cap / capacity.
GET <code>/api/physicians/directory</code>	Auth	Org-wide provider directory (HIPAA-safe fields).
POST <code>/api/round-robin/reset</code>	Director	Reset the round-robin rotation index.

5.3 Patients & assignments

Method & path	Auth	Purpose
POST /api/patients	ER physician	Register a new patient.
GET /api/patients	Auth	List patients in the organization.
POST /api/assignments	ER physician	Create an assignment request (round-robin or manual).
GET /api/assignments/pending	Hospitalist	Pending assignment queue.
GET /api/assignments/my	Hospitalist	Active/accepted assignments.
PATCH /api/assignments/:id/override	Director	Force-override an assignment.
PATCH /api/assignments/:id/reassign	Director / ER	Trigger reassignment logic.
POST /api/assignments/:id/resend-sms	Director	Re-send the assignment SMS notification.

5.4 Messaging

Method & path	Auth	Purpose
GET /api/messaging/conversations	Auth	List the caller's conversations.
POST /api/messaging/conversations	Auth	Start a direct or group conversation.
POST /api/messaging/send	Auth	Send a message into a conversation.
GET /api/messaging/conversations/:id/messages	Participant	Fetch message history.
POST /api/messaging/messages/mark-read	Auth	Batch mark messages read.
DELETE /api/messaging/messages/:messageId	Sender	Delete a message (audited).

5.5 Organizations, developer, CMS & SMS

Method & path	Auth	Purpose
GET /api/organizations/:code	Public	Look up an organization by short code.
GET /api/dev/organizations	Developer	List all tenants.
POST /api/dev/organizations	Developer	Create a tenant.
DELETE /api/dev/organizations/:id	Developer	Delete a tenant (DEFAULT protected).
POST /api/dev/impersonate	Developer	Impersonate a user for debugging.
POST /api/dev/ai-diagnostics	Developer	Run AI-assisted system diagnostics.
GET /api/landing-settings	Public	Fetch CMS landing content.
PUT /api/dev/landing-settings	Developer	Update CMS landing content.

Method & path	Auth	Purpose
POST /api/admin/twilio/config	Developer	Configure per-org Twilio credentials.
GET /api/sms-history	Director	Audit of sent SMS notifications.

5.6 Health & mobile

Method & path	Auth	Purpose
GET /api/health	Public	System/DB health probe.
GET /api/qr-test, /api/qr-display	Auth	QR code for mobile onboarding (encodes the /m/ proxy URL).
GET /api/mobile/messages	Auth	Mobile-optimized message fetch.
GET /api/mobile/er-admissions	Auth	Mobile view of incoming patients.

5.7 WebSocket server

A WebSocket server is mounted at /ws. On connection the server parses the `docturn.sid` cookie, unsigns it, and looks up the session row to resolve `passport.user`. Connections that fail session resolution or have no valid organization are closed with code 1008. Each socket is stored in a clients map keyed by user, carrying its `userId` and `organizationId` so that all fan-out stays tenant-scoped.

Message type	Direction	Meaning
CONNECTION_ESTABLISHED	server → client	Handshake acknowledgement with connection id.
USER_PRESENCE_CHANGED	server → org	A user connected/disconnected (org-scoped).
typing_start / typing_stop	client → server	Typing intent for a conversation.
user_typing	server → participants	Typing indicator relayed to conversation members.
MESSAGE_RECEIVED	server → users	New message / delivery acknowledgement.
PROVIDER_CREATED	server → org	A new provider was created in the tenant.
HOSPITALIST_DELETED	server → org	A provider was removed from the tenant.
PING / PONG	both	20-second heartbeat that also updates presence.

Two helpers govern delivery: `broadcast(message, organizationId)` fans out to every socket in a tenant, and `sendToUsers(userIds, message)` delivers to specific participants (e.g. the two members of a direct conversation).

6. Authentication & Authorization

6.1 Authentication

Authentication uses Passport with a local username/password strategy. Login is organization-aware: credentials are validated against the user record within the resolved organization.

- **Password hashing** — `scrypt` (`crypto.scrypt`) with per-user salt; legacy `bcrypt` hashes are still verified for backward compatibility.
- **Session store** — server-side sessions persisted in PostgreSQL; the cookie carries only the signed session id.
- **Cookie** — `docturn.sid`, `httpOnly`; in production `secure` and `sameSite=strict`.
- **Session lifetime** — 15-minute max age in production (HIPAA), 24 hours in development; rolling resets expiry on activity.
- **Registration** — self-registration creates a pending record; a director must approve and assign a role before the account can authenticate.

6.2 Multi-factor authentication

MFA is handled by a dedicated two-factor service and supports both TOTP (authenticator apps via `speakeasy`) and SMS one-time codes (via Twilio).

- On login with MFA enabled, the server returns 202 with `twoFactorRequired: true` and holds a pending user id in the session.
- The challenge is completed at `/api/2fa/complete-login`, which finalizes the authenticated session.
- Ten one-time backup codes are generated at TOTP setup and stored as an array; each may be redeemed once if the primary device is unavailable.

6.3 Role-based access control

Authorization is enforced server-side in route handlers and mirrored in the client for UX. The web client wraps protected routes in a `ProtectedRoute` component, but the server remains the source of truth.

- Roles: `director`, `er_director`, `er_doctor/er_physician`, `hospitalist`, `developer`.
- Each handler checks the caller's role and that the target record's `organizationId` matches the caller's (developers may act cross-tenant).
- A HIPAA compliance service centralizes permission semantics such as `view_all`, `create_assignments`, and `view_assigned`.
- Configurable `role_types` allow per-tenant permission and portal-access definitions stored as JSON.

6.4 Replit OIDC note

An OIDC strategy exists in `server/replitAuth.ts` as an alternative login path, but it is not the primary mechanism wired into the running application; the local Passport strategy in `server/auth.ts` governs production login.

7. Multi-Tenancy

Tenancy is the platform's central design constraint. Every clinically or administratively significant table is scoped by `organizationId`, and the storage layer exposes organization-aware query methods so that handlers cannot accidentally read or write across tenants.

- **Isolation** — storage methods such as `getWorkingHospitalistsByOrganization` and `getActivePatientsForOrganization` always filter by tenant.
- **Per-tenant configuration** — round-robin shift types and timeouts, branding/CMS, and feature flags live in `organizations`, `portal_configurations`, and `CMS` tables.
- **Per-tenant integration credentials** — Twilio, Firebase, and Amion credentials are stored per organization rather than globally.
- **Real-time scoping** — WebSocket broadcasts are keyed by `organizationId`; presence and status never leak across tenants.
- **Cross-tenant operations** — reserved for the developer role through explicit `/api/dev/**` endpoints.

The round-robin selector is called out in the threat model as especially sensitive: a selector that ignored tenant scope could silently route a patient to a provider in another hospital. Rotation helpers therefore operate strictly within a single organization.

8. Patient Assignment Workflow

The assignment workflow is the operational heart of DocTurn. It connects ER intake to hospitalist acceptance through automated routing and multi-channel notification.

8.1 End-to-end flow

- **Intake** — an ER physician submits a patient (optionally pasting free-text notes that OpenAI converts into structured fields and a suggested specialty).
- **Routing** — the server selects a hospitalist via round-robin or accepts a manual selection.
- **Persistence** — a patients row and an assignments row (status pending, with expiresAt) are created.
- **Notification** — the provider is notified by WebSocket, Firebase push, and/or Twilio SMS.
- **Acknowledgement** — the provider accepts or declines; acceptance moves the status to accepted and updates census, decline/expiry triggers re-routing.

8.2 Round-robin rotation

Rotation logic lives in the storage layer (`getNextHospitalistInRotationByOrganization`) and the routes layer.

- **Eligibility** — candidates must be active today, match the required shift type, and be below their `patientCap`.
- **Selection** — the standard path favors the lowest current patient count; a rotation-index path (persisted per organization in `system_settings`) cycles providers sequentially.
- **Cap relief** — if every eligible provider is at capacity, each provider's `patientCap` is incremented by one so the queue can still drain.

8.3 Expiration & reassignment

When an assignment is created the server computes `expiresAt` from `getAssignmentTimeout(organizationId)`. If the provider does not respond before that time, the assignment is treated as expired and becomes eligible for automatic or director-initiated reassignment to the next provider in rotation.

9. External Integrations

All third-party calls are made server-side. Credentials are either environment secrets or, for tenant-specific services, stored per organization in the database.

9.1 Twilio (SMS & MFA)

- **Secrets** — TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_PHONE_NUMBER (alias forms supported).
- **Use** — assignment notifications and SMS-based MFA codes; SMS is the fallback channel when push is unavailable.
- **Failure handling** — send errors are caught and logged; the workflow continues and falls back to other channels.

9.2 OpenAI (AI assistance)

- **Secrets** — AI_INTEGRATIONS_OPENAI_API_KEY and AI_INTEGRATIONS_OPENAI_BASE_URL (Replit integration; OPENAI_API_KEY also supported).
- **Patient extraction** — gpt-4o turns free-text intake into structured clinical fields and a suggested specialty.
- **Command processing** — gpt-4o-mini interprets natural-language commands (e.g. assignment requests) into system actions.
- **Safeguards** — PHI is truncated in prompts; if extraction fails, raw text is used as the chief complaint.

9.3 Firebase (push notifications)

- **Config** — per-organization project id and service-account credentials stored in `firebase_configurations`.
- **Use** — HIPAA-safe push for new assignments and messages; payloads are generic ("New secure message — open app to view").
- **Failure handling** — if push fails or no device tokens exist, the system falls back to SMS with a secure web link.

9.4 Amion (scheduling)

- **Config** — per-organization `amionFacilityId` and `amionApiKey` in organizations.
- **Use** — optional sync of on-call providers and shifts.
- **Failure handling** — falls back to locally managed schedules when unreachable or unconfigured.

10. Frontend Architecture

10.1 Web SPA

- **Routing** — wouter in `client/src/App.tsx`; role-specific routes for director, ER doctor, hospitalist, and developer dashboards plus the public auth/landing surfaces.
- **Server state** — TanStack Query v5 with a shared `queryClient`; the default query function fetches by query key, and `apiRequest` wraps mutations.
- **Auth & realtime context** — `AuthProvider` exposes the session; `WebSocketProvider` streams real-time events with a 20-second heartbeat.
- **Design system** — Tailwind CSS + `shadcn/ui` primitives, light/dark theming, and a mobile-first responsive layout.
- **PWA** — a manifest and a network-first service worker keep clinical data current while caching static assets for offline fallback.

The `apiRequest` helper attaches credentials (or a bearer test token in the multi-session test harness), serializes JSON, and throws on any non-2xx response so that TanStack Query mutation error states fire correctly.

10.2 Role dashboards

Dashboard	Focus
Director	Provider management, shift-type and round-robin configuration, drag-and-drop rotation ordering, reassignment.
ER doctor	Patient intake, quick (round-robin) assign, and manual assign to a specific hospitalist.
Hospitalist	Census management and accept/decline of incoming assignment requests.
Developer	Cross-tenant administration, user impersonation, system logs, CMS, and the AI monitor.

10.3 Mobile app

- **Framework** — Expo SDK 52 / React Native 0.76 with bottom-tab navigation (Dashboard, Messages, Care Team, Directory, Profile).
- **Shared backend** — a singleton `ApiClient` mirrors the web `apiRequest` (including session cookie handling) against the same API.
- **Realtime** — a native `WebSocket` service implements the same message protocol as the web client.
- **Sessions** — the same PostgreSQL-backed session model lets a user move between web and mobile seamlessly.
- **Dev access** — an Express `/m/` proxy serves the Expo web build (internal port 8080) through the single externally exposed port.

11. Security & HIPAA Compliance

11.1 Transport & HTTP hardening

- TLS terminates at the platform edge; production cookies are secure and `sameSite=strict`.
- Helmet sets standard security headers (HSTS, X-Content-Type-Options, X-Frame-Options, Referrer-Policy). The CSP is relaxed in development for Vite and should be tightened in production.
- Tiered rate limiting: stricter limits on auth routes (≈ 100 / 15 min) and a broader API limit (≈ 1000 / 15 min).

11.2 Audit & PHI logging

- **Audit logs** — authentication events, user management, assignment operations, CMS edits, and conversation deletions are recorded with actor, resource, details, IP, user agent, and risk level.
- **PHI access logs** — a dedicated trail records which patient and which data fields were accessed and how.
- **Middleware** — HIPAA audit and PHI-access middleware automatically log activity on sensitive endpoints; response-body logging in the server endpoint is treated as security-sensitive because it can capture API payloads.

11.3 Session & account controls

- Short (15-minute) production session timeout with rolling renewal and server-side inactivity enforcement.
- MFA (TOTP + SMS) with one-time backup codes.
- Director approval gate for new account activation.

11.4 Threat-model focus areas

- **Broken access control** across the very large route surface — every route must assert role and tenant.
- **Cross-tenant leakage** from a missing `organizationId` filter — especially in rotation/selection helpers.
- **Production reachability of dev/test helpers** — quick-login, impersonation, and self-test endpoints must be unavailable to production users.
- **Information disclosure** — responses, logs, and diagnostics must never expose password hashes, session ids, backup codes, secrets, or PHI.

See `threat_model.md` in the repository for the full asset inventory, trust boundaries, and STRIDE breakdown.

12. Deployment & Operations

12.1 Environments

- **Development** — Replit workflows run the web app (`npm run dev`, port 5000) and the Expo web build (port 8080) side by side against one Postgres database.
- **Production** — multi-stage Docker build, Docker Compose, Nginx reverse proxy, HTTPS, and auto-scaling.

12.2 Configuration & secrets

Variable	Purpose
DATABASE_URL	PostgreSQL connection string.
SESSION_SECRET	Signing secret for the session cookie.
TWILIO_ACCOUNT_SID / AUTH_TOKEN / PHONE_NUMBER	SMS and SMS-MFA.
AI_INTEGRATIONS_OPENAI_API_KEY / _BASE_URL	OpenAI access via the Replit integration.
Firebase service account (per org)	Push notifications (stored in DB, not env).
Amion facility id / API key (per org)	Scheduling integration (stored in DB).

12.3 Schema changes

- Schema is the source of truth in `shared/schema.ts`; the storage layer is updated to match.
- Changes are applied with `npm run db:push` (or `--force` when a safe data-loss prompt appears). No hand-written SQL migrations.

12.4 Observability

- Structured application logging plus the HIPAA audit and PHI trails.
- A `/api/health` probe for liveness/DB checks.
- A developer AI-monitoring dashboard surfacing real-time metrics, insights, and diagnostics.

13. Appendix

13.1 Repository layout

Path	Contents
shared/schema.ts	Drizzle data model + Zod insert schemas (source of truth).
server/index.ts	Express entrypoint: middleware, rate limiting, server bootstrap.
server/routes.ts	REST API + WebSocket server (largest module).
server/auth.ts	Passport setup, sessions, registration, MFA login flow.
server/storage.ts	IStorage interface + Drizzle-backed DatabaseStorage (tenant-scoped).
server/*-service.ts	AI, messaging, two-factor, Firebase, presence services.
client/src/	React web SPA (pages, components, lib, hooks).
mobile-app/	Expo React Native client sharing the backend.
threat_model.md	Security architecture, assets, trust boundaries, STRIDE.

13.2 Glossary

Term	Definition
Tenant / organization	An independent hospital whose data is isolated by organizationId.
Assignment	A request routing a patient to a hospitalist, with a lifecycle status.
Round-robin	The rotation algorithm that selects the next eligible hospitalist.
Hospitalist	A provider who receives and manages admitted patients.
PHI	Protected Health Information — patient-identifying clinical data.
MFA	Multi-factor authentication (TOTP or SMS) with backup codes.
CMS	Content management for the public landing/contact pages.

End of document.